



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

**Wydział Informatyki, Elektroniki i Telekomunikacji
Instytut Telekomunikacji**

Kryptografia postkwantowa

***Implementacja algorytmu kryptografii krzywych eliptycznych
na przykładzie ECDSA - raport***

Aleksander Brachman, Kamil Maciaszek, Krzysztof Skociński

Spis treści:

I.	Wprowadzenie	2
II.	Porównanie ECDSA z RSA.....	4
III.	Wymagania funkcjonalne i нефункционалне wobec aplikacji	5
IV.	Techniczna architektura aplikacji.....	6
V.	Wyniki działania aplikacji oraz porównanie z gotową implementacją	7
VI.	Podsumowanie	10

I. Wprowadzenie

1. Podstawy matematyczne ECDSA

- **Równanie krzywej eliptycznej:** Podstawą ECDSA są krzywe eliptyczne zdefiniowane równaniem:

$$y^2 = x^3 + ax + b \pmod{p}$$

gdzie: **a, b, p** to parametry. **Parametr p jest liczbą pierwszą.**

- Punkty na krzywej tworzą grupę addytywną. W tej grupie możliwe jest wykonywanie takich operacji jak dodawanie punktów i mnożenie skalarnie (wielokrotne dodawanie).
- Bezpieczeństwo zapewnia **problem dyskretnego logarytmu na krzywych eliptycznych (ECDLP)**: Dla danych punktów **P** i **Q = kP** obliczenie **k** (mnożnika skalarnego) jest obliczeniowo niewykonalne w czasie wielomianowym przy użyciu obecnych algorytmów, jeśli **k** jest duże i wybrane losowo.

2. Kluczowe elementy ECDSA

- **Klucz prywatny (d):** Losowo wybrana liczba całkowita z dużego zakresu.
- **Klucz publiczny (Q):** Punkt na krzywej eliptycznej, obliczany jako **Q = dP**, gdzie **P** jest publicznie znanym punktem bazowym (generatorem).

3. Proces podpisywania wiadomości

Proces podpisywania pozwala, przy użyciu klucza prywatnego, na wygenerowanie podpisu wiadomości, który zapewnia, że:

- wiadomość pochodzi od właściwego nadawcy,
- nadawca nie może zaprzeczyć wysłaniu przez siebie wiadomości,
- treść wiadomości nie uległa zmianie podczas jej transmisji do odbiorcy.

Tylko właściciel klucza prywatnego może wygenerować podpis. Proces podpisywania wiadomości w algorytmie ECDSA przebiega następująco:

- A. **Oblicz hash wiadomości:** Oblicz hash (z) wiadomości, aby uzyskać ciąg znaków o stałym rozmiarze.
- B. **Wygeneruj losowy skalar:** Wybierz losową liczbę całkowitą k (dla każdej wiadomości) z zakresu $[1, n-1]$, gdzie n to rząd punktu bazowego P .
- C. **Mnożenie punktu:** Oblicz $R = kP$, gdzie $R = (x_R, y_R)$.
- D. **Wyznaczenie składników podpisu:**
 - a. $r = x_R \bmod n$ (upewnij się, że $r \neq 0$).
 - b. $s = k^{-1}(z + rd) \bmod n$, gdzie k^{-1} to odwrotność modularna liczby k .
- E. **Podpis:** Podpis to para wyznaczonych wartości (r, s) .

4. Proces weryfikacji podpisu

Weryfikacja polega na sprawdzeniu, czy podpis pasuje do wiadomości i czy został wygenerowany przez właściciela klucza prywatnego.

- A. **Oblicz hash wiadomości:** Compute the hash (z) of the message.
- B. **Oblicz wartości pośrednie:**
 - a. $w = s^{-1} \bmod n$ (odwrotność modularna s).
 - b. $u_1 = zw \bmod n$.
 - c. $u_2 = rw \bmod n$.
- C. **Mnożenie punktów i dodawanie:**
 - a. Oblicz $X = u_1P + u_2Q$, gdzie $X = (x_X, y_X)$.
- D. **Zwaliduj podpis:**
 - a. Jeśli $r = x_X \bmod n$, podpis jest prawidłowy.

5. Dlaczego to działa?

Proces weryfikacji opiera się na relacji między kluczami publicznym i prywatnym:

- Podczas podpisywania składnik s zawiera $k^{-1}(z + rd) \bmod n$, co wiąże z (hash wiadomości) z kluczem prywatnym d .
- Podczas weryfikacji, właściwości grupy krzywej eliptycznej oraz publiczny charakter P pozwalają na odtworzenie równania $R = kP$ bez znajomości k lub d .
- Ponieważ P , Q oraz parametry krzywej są publiczne, każdy może sprawdzić poprawność podpisu, ale tylko właściciel klucza prywatnego może go utworzyć.

6. Bezpieczeństwo ECDSA

Bezpieczeństwo ECDSA opiera się na:

- Trudności rozwiązania **ECDLP** w praktycznym czasie.
- Właściwym doborze parametrów krzywej eliptycznej, aby uniknąć podatności, takich jak ataki na małe podgrupy.
- Silnej losowości przy wyborze k . Wielokrotne użycie tego samego k w podpisach prowadzi do ujawnienia klucza prywatnego.

II. Porównanie ECDSA z RSA

1. Podstawy matematyczne

ECDSA	RSA
Oparte na problemie dyskretnego logarytmu na krzywych eliptycznych (Elliptic Curve Discrete Logarithm Problem, ECDLP)	Oparte na trudności faktoryzacji iloczynu dwóch dużych liczb pierwszych (Integer Factorization Problem, IFP)
Działa na grupach krzywych eliptycznych	Operuje na arytmetyce modularnej z dużymi liczbami całkowitymi

2. Długość klucza i bezpieczeństwo

ECDSA	RSA
Zapewnia wyższy poziom bezpieczeństwa na bit długości klucza. Klucz ECDSA o długości 256 bitów jest w przybliżeniu równoważny pod względem bezpieczeństwa kluczowi RSA o długości 3072 bitów	Wymaga znacznie większych kluczy dla równoważnego poziomu bezpieczeństwa. Standardem jest klucz RSA o długości 2048 bitów, ale w przyszłości konieczne będzie stosowanie kluczy o długości 3072 bity lub większych
Mniejsze klucze sprawiają, że ECDSA jest bardziej wydajne pod względem pamięci, przepustowości i obliczeń	Większe klucze wymagają więcej miejsca na ich przechowywanie oraz większej mocy obliczeniowej

3. Wydajność

ECDSA	RSA
Szybsze podpisywanie: Operacje podpisywania w ECDSA są zazwyczaj szybsze niż w RSA	Szybsza weryfikacja: Weryfikacja podpisów w RSA jest zazwyczaj szybsza niż operacje podpisywania
Generowanie i weryfikacja podpisów są wydajne obliczeniowo dzięki mniejszym rozmiarom kluczy i operacjom na krzywych eliptycznych	Generowanie kluczy oraz szyfrowanie/deszyfrowanie są wolniejsze ze względu na wykładnicze operacje modularne z dużymi liczbami całkowitymi

4. Rozmiar podpisu

ECDSA	RSA
Krótsze podpisy: Rozmiar podpisu jest mniejszy, np. 64 bajty dla klucza o długości 256 bitów	Dłuższe podpisy: Rozmiar podpisu odpowiada długości klucza, np. 256 bajtów dla klucza o długości 2048 bitów
Krótsze podpisy są korzystne w aplikacjach o ograniczonej przepustowości (np. IoT, urządzenia mobilne)	Dłuższe podpisy mogą wymagać większej przepustowości i miejsca w nośnikach danych

5. Zastosowania

ECDSA	RSA
Powszechnie stosowane w nowoczesnych systemach, zwłaszcza tych z ograniczonymi zasobami (np. IoT, urządzenia mobilne, blockchajny takie jak Bitcoin i Ethereum)	Używane w tradycyjnych systemach (np. HTTPS, szyfrowanie e-maili, VPN)
Często używane w protokołach takich jak TLS, SSH oraz w kryptowalutach do tworzenia kompaktowych podpisów cyfrowych	Nadal szeroko stosowane w systemach starszego typu oraz ze względów kompatybilności

6. Zarządzanie kluczem

ECDSA	RSA
Mniejsze klucze sprawiają, że ECDSA jest bardziej praktyczne w środowiskach z ograniczoną pojemnością pamięci lub przepustowością transmisji	Używane w tradycyjnych systemach (np. HTTPS, szyfrowanie e-maili, VPN)

7. Odporność na ataki ze strony komputerów kwantowych

Oba algorytmy są podatne na ataki ze strony komputerów kwantowych, np. przez wykorzystanie algorytmu Shora. W przyszłości konieczne będzie zastąpienie ECDSA oraz RSA post-kwantowymi metodami kryptograficznymi.

III. Wymagania funkcjonalne i нефункционаłne wobec aplikacji

Głównym celem projektu było napisanie prostej aplikacji w języku Python, która implementuje krok po kroku proces podpisywania i weryfikacji wiadomości, zgodnie z algorytmem ECDSA. Poniżej przedstawiono główne wymagania funkcjonalne i нефункционаłne wobec aplikacji:

1. Wymagania funkcjonalne

- Implementacja algorytmu ECDSA:
 - Generowanie kluczy prywatnych i publicznych zgodnie z zasadami kryptografii krzywych eliptycznych.
 - Generowanie podpisów cyfrowych poprzez proces podpisywania kluczem prywatnym.
 - Ewaluacja danych podpisanych cyfrowo poprzez proces dołączania do nich klucza publicznego.
- Obsługa podstawowych właściwości krzywych eliptycznych
 - Do wyboru musi być dostępna co najmniej jedna standardowa krzywa zgodna z NIST.

- Kontrola informacji wejściowych
 - Algorytm powinien pozwalać na wpisanie przez użytkownika wiadomości, którą chce podpisać oraz sprawdzać czy wiadomość np. nie jest pusta.
- Kontrola poprawności informacji wejściowych
 - Algorytm podczas procesu weryfikacji podpisu powinien informować użytkownika, czy podane informacje wejściowe składają się na poprawny lub błędny podpis.

2. Wymagania niefunkcjonalne

- Efektywność obliczeniowa
 - Oczekuje się, że algorytm będzie spełniał rozsądne czasy pracy dla kluczy o różnych długościach.
 - Optymalizacja najważniejszych operacji typu mnożenie punktów krzywej eliptycznej.
- Jakość kodu
 - Kod algorytmu ECDSA powinien być napisany w sposób zrozumiały oraz odpowiednio dokumentowany, aby przybliżał kwestie matematyczne, które są wyznaczone przez specyfikę tego algorytmu.
- Bezpieczeństwo
 - Algorytm powinien generować klucze w sposób kryptograficznie bezpieczny (np. wykorzystanie secrets lub os.urandom)
 - Odpowiednie zarządzanie kluczami prywatnymi (np. nie przechowywanie ich w postaci jawnej w plikach).
- Modularność
 - Kod powinien być podzielony na moduły (np. generowanie kluczy, podpisywanie, weryfikacja), co ułatwi testowanie i rozszerzanie.
- Walidacja
 - Aplikacja powinna być przetestowana na różnych zestawach danych (np. krótkie i długie wiadomości) oraz przy użyciu różnych krzywych eliptycznych.

IV. Techniczna architektura aplikacji

Do napisania aplikacji wybrano język programowania Python wraz z podstawowymi bibliotekami oferowanymi przez ten język. Aplikacja została napisana i przetestowana w Google Colab. Komunikacja między użytkownikiem, a aplikacją odbywa się za pośrednictwem terminalu. Szczegółowy opis utworzonej aplikacji przedstawiono w dokumentacji technicznej do projektu.

Zaimplementowane krzywe eliptyczne w aplikacji to krzywe eliptyczne, które są standaryzowane przez NIST - krzywe p224, p256, p384, p521. Parametry poszczególnych

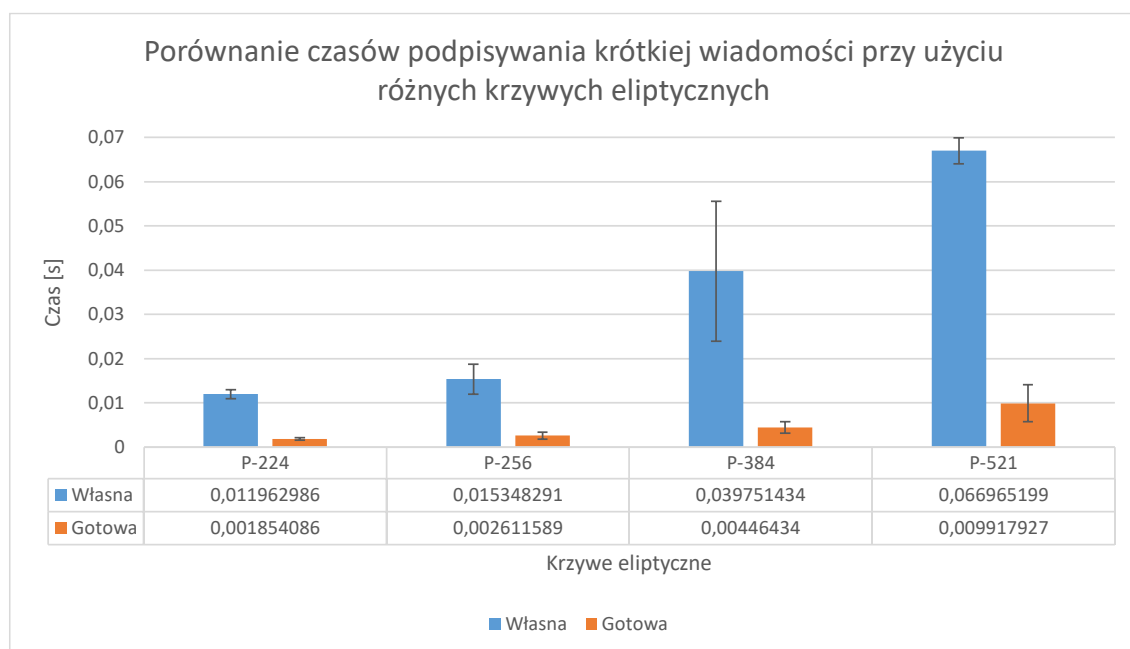
krzywych w aplikacji zostały ustawione zgodnie z zaleceniami NIST (<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-186.pdf>).

Istotą bezpiecznego procesu generowania podpisu w ECDSA jest użycie bezpiecznego generatora liczb pseudolosowych. W ramach implementacji aplikacji zdecydowano się na użycie biblioteki *secrets.py*, która jest przeznaczona do generowania kryptograficznie silnych liczb losowych w środowisku Python.

V. Wyniki działania aplikacji oraz porównanie z gotową implementacją

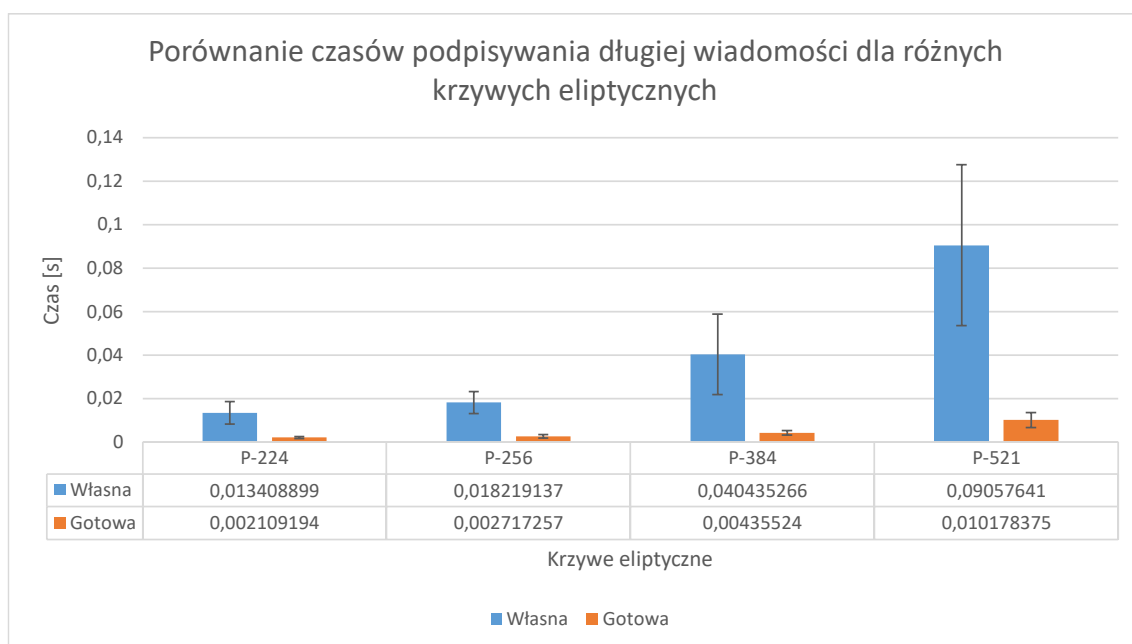
W celu zbadania wydajności procesów podpisywania wiadomości i jej weryfikacji, w zaimplementowanej w ramach projektu aplikacji, wykonano serie pomiarów czasów tychże procesów dla dwóch różnych długości wiadomości i czterech krzywych, które zostały zdefiniowane w aplikacji. Podobne badania wykonano również przy wykorzystaniu gotowej implementacji ECDSA w Pythonie (biblioteka *ecdsa*) w ramach porównania “klasycznej”, własnej implementacji ECDSA do zoptymalizowanej biblioteki.

Podpisanie wiadomości krótkiej (wiadomość “test”) – rys. 1.



Rys. 1. Czasy podpisywania krótkiej wiadomości z wykorzystaniem algorytmu ECDSA.

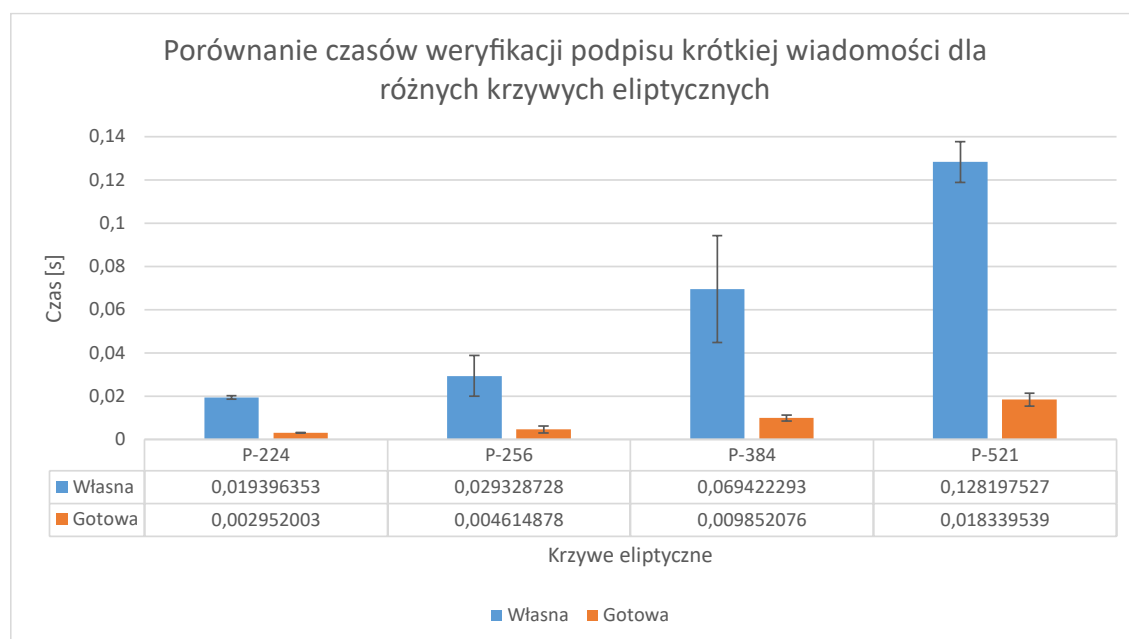
Podpisanie wiadomości długiej (wiadomość zawierająca około 3500 znaków) – rys. 2.



Rys. 2. Czasy podpisywania długiej wiadomości z wykorzystaniem algorytmu ECDSA.

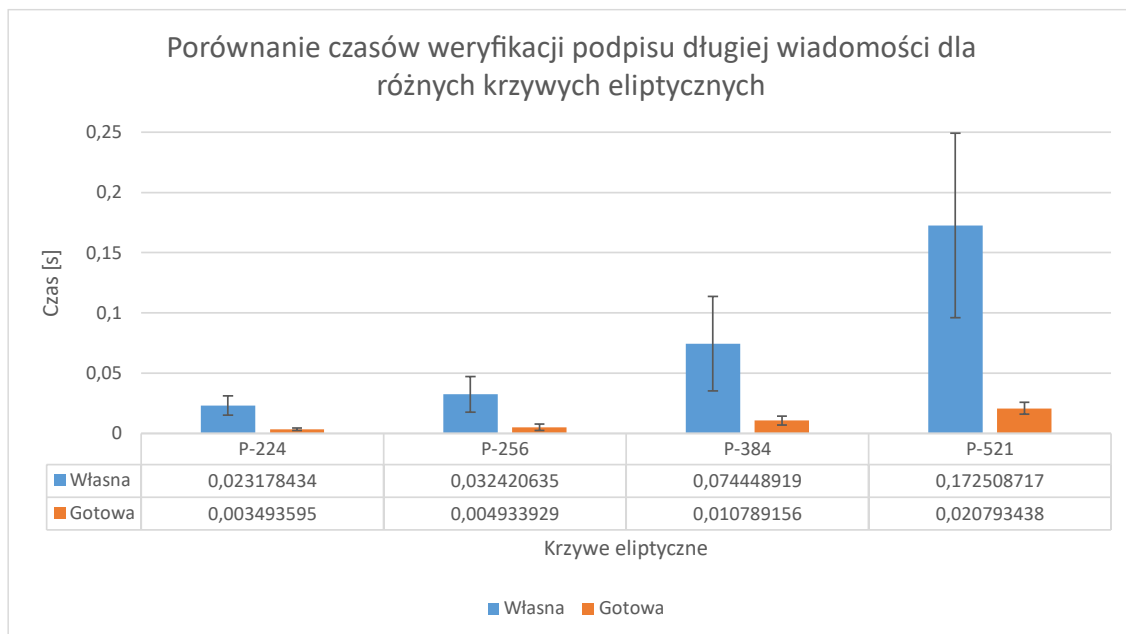
Zarówno dla krótszej, jak i dłuższej wiadomości średni czas procesu podpisywania wiadomości przy użyciu własnej implementacji ECDSA jest kilkukrotnie dłuższy niż przy skorzystaniu z gotowej implementacji. W przypadku krótszej wiadomości gotowa implementacja jest szybsza średnio 7.1 razy (średnia dla czterech krzywych), a w przypadku dłuższej wiadomości – 8.4 razy. Można również zauważyć, że czas podpisu zwiększa się (dla obu implementacji) wraz z zwiększaniem się długości generowanego podpisu w bitach.

Weryfikacja podpisu wiadomości krótkiej – rys. 3.



Rys. 3. Czasy weryfikacji podpisu krótkiej wiadomości z wykorzystaniem algorytmu ECDSA.

Weryfikacja podpisu wiadomości długiej – rys. 4.



Rys. 4. Czasy weryfikacji podpisu długiej wiadomości z wykorzystaniem algorytmu ECDSA.

Podobnie jak dla procesu podpisywania wiadomości, proces weryfikacji podpisu wiadomości wykonywany jest szybciej przy użyciu gotowej implementacji, dla obu długości wiadomości – średnio 6.9 razy dla krótkiej wiadomości oraz 7.6 razy dla długiej wiadomości. W celu wyjaśnienia tej różnicy przeprowadzono analizę statyczną kodu gotowej implementacji z której wynika, że zawiera ona bazę wyników niektórych obliczeń, związanych z mnożeniem punktu przez skalar, z której pobiera odpowiedzi zamiast przeliczać je na nowo, co zwiększa szybkość algorytmu.

Zwiększanie się długości w bitach weryfikowanego podpisu wpływa na zwiększanie się czasu procesu weryfikacji podpisu.

Analizując uzyskane wyniki można stwierdzić, iż czasy wykonywania weryfikacji podpisu są dłuższe od czasów podpisywania wiadomości. Dla krótkiej wiadomości, przy użyciu własnej implementacji ECDSA, podpis odbywa się średnio 1.83 razy szybciej niż weryfikacja (średnia dla czterech krzywych), a przy użyciu gotowej implementacji – 1.89 razy. Bardzo zbliżone wyniki zostały wyznaczone dla długiej wiadomości; w tym przypadku podpis, przy użyciu własnej implementacji ECDSA, odbywa się średnio 1.86 razy szybciej niż weryfikacja, a przy skorzystaniu z gotowej implementacji – 2.07 razy szybciej. Dzieje się tak, pomimo tego, że obie operacje mają taką samą złożoność czasową $O(n \log n)$, ponieważ operacja dominująca, jaką jest mnożenie punktu przez skalar, jest podczas weryfikacji wykonywana dwukrotnie, a podczas podpisywania wykonuje się jedynie raz.

Dodatkowo, w ramach badań nad własną implementacją algorytmu ECDSA, zmierzono czasy wykonywania poszczególnych operacji składających się na proces podpisywania wiadomości i weryfikacji podpisu wiadomości, które przedstawiono w tab. 1.

Tab. 1. Czasy wykonywania poszczególnych operacji matematycznych w własnej implementacji (wiadomość "test", krzywa: P-256)

Operacja	Czas [s]	Uwagi do operacji
Klucz prywatny d	0,00003981590270996094	Losowanie liczby z zbioru
Klucz publiczny Q	0,011663675308227539	Mnożenie punktu przez skalar
Skrót wiadomości (podpis)	0,00014853477478027344	Funkcja skrótu
Liczba k (podpis)	0,000022172927856445312	Losowanie liczby z zbioru
Punkt P (podpis)	0,011980056762695312	Mnożenie punktu przez skalar
Modulo współrzędnej x punktu P	0,0000019073486328125	Redukcja modulo
Wartość s podpisu	0,00004887580871582031	Odwrotność modularna
Skrót wiadomości (weryfikacja)	0,000019073486328125	Funkcja skrótu
Wartość w (weryfikacja)	0,00005745887756347656	Odwrotność modularna
Wartość u1 (weryfikacja)	0,0000026226043701171875	Iloczyn + modulo
Wartość u2 (weryfikacja)	0,0000016689300537109375	Iloczyn + modulo
Punkt u1G (weryfikacja)	0,018656015396118164	Mnożenie punktu przez skalar
Punkt u2Q (weryfikacja)	0,01350712776184082	Mnożenie punktu przez skalar
Dodawanie u1G i u2Q (weryfikacja)	0,00005626678466796875	Dodawanie punktów na krzywej
Decyzja czy podpis jest prawidłowy	0,000001430511474609375	Redukcja modulo

Najdłuższe czasy uzyskano dla operacji, których podstawą jest mnożenie punktu przez skalar. Operacje te są dłuższe o setki, a nawet tysiące razy, w porównaniu do innych obliczeń wykonywanych w implementacji.

VI. Podsumowanie

W ramach projektu udało się przygotować niezbędną teorię, która pomaga w zrozumieniu działania algorytmu ECDSA i jego zalet oraz udało się zrealizować prostą implementację algorytmu w języku Python. W ramach naszego rozwiązania zaprogramowano każdą operację matematyczną składającą się na proces podpisu wiadomości i jej weryfikacji w ECDSA. Aplikacja działa prawidłowo, tzn. umożliwia wybór krzywej na której użytkownik chce pracować, generuje klucze: prywatny oraz publiczny, umożliwia wprowadzenie przez użytkownika wiadomości, którą chce podpisać oraz weryfikuje podpis.

Badania nad własną implementacją wykazały, iż procesy podpisywania wiadomości i jej weryfikacji trwają w rozsądnym, akceptowalnym czasach, chociaż są one kilkakrotnie dłuższe

od czasów uzyskiwanych dla gotowej implementacji, przy wykorzystaniu biblioteki `ecdsa` w Pythonie. Analiza wydajnościowa operacji własnej implementacji wykazała, że operacje, których podstawą jest mnożenie punktu przez skalar, wykonują się zdecydowanie dłużej, w porównaniu z pozostałymi operacjami. Dwukrotna obecność operacji mnożenia punktu przez skalar wpływa również na wydłużenie procesu weryfikacji podpisu względem procesu generowania podpisu.

Wpływ na uzyskiwane czasy ma również krzywa eliptyczna, którą wybieramy do procesu podpisu i weryfikacji. Im dłuższe (w bitach) parametry liczbowe krzywej, tym dłuższy staje się czas ww. procesów. Z drugiej strony, używanie dłuższych kluczy zwiększa bezpieczeństwo algorytmu ECDSA.

W ramach dalszych prac nad aplikacją należałoby zastanowić się nad optymalizacją zaprogramowanych funkcji celem zmniejszenia różnicy w wydajności pomiędzy własną implementacją, a gotową biblioteką. Dodatkową, pożądaną funkcjonalnością aplikacji w przyszłości byłaby możliwość cyfrowego podpisywania plików (np. pdf lub docx) z wykorzystaniem algorytmu ECDSA.

Końcowy podział zadań w ramach projektu:

- 1) Przygotowanie wstępu teoretycznego - teoria stojąca za ECDSA, zastosowania ECDSA, zalety i wady w porównaniu z innymi metodami kryptograficznymi (np. vs RSA) - **Kamil Maciaszek**
- 2) Zdefiniowanie wymagań funkcjonalnych i niefunkcjonalnych wobec aplikacji - **Krzysztof Skociński**
- 3) Techniczna architektura ECDSA w aplikacji – wybrany język programowania, wybrane krzywe eliptyczne (wraz z ich parametrami), wybrany generator liczb losowych itd. - **Kamil Maciaszek**
- 4) Implementacja aplikacji (napisanie kodu) - **Aleksander Brachman, Kamil Maciaszek, Krzysztof Skociński**
- 6) Przeprowadzenie testów na aplikacji w celu zmierzenia szybkości algorytmu i porównanie z istniejącą implementacją innego algorytmu - **Krzysztof Skociński, Aleksander Brachman**
- 7) Analiza złożoności czasowej kodu – **Kamil Maciaszek**
- 8) Wykonanie dokumentacji technicznej - **Aleksander Brachman**
- 9) Utworzenie raportu z wykonanego projektu - **Aleksander Brachman, Kamil Maciaszek**